Smart MCQ Solver Challenge
Kaggle Competition Report

DL -GENAI - T2 2026

Submitted by
Mohamed Shamil MA
23f3003322

Introduction

Problem Statement

The **Smart MCQ Solver Challenge** is a Kaggle competition focused on developing Artificial Intelligence (AI) systems capable of solving challenging multiple-choice questions (MCQs). Each question consists of a textual prompt and five answer options labeled **A, B, C, D,** and **E**. Rather than predicting only a single correct answer, we are required to rank the **top three most probable answers** for every question.

The competition aims to evaluate a model's ability to understand natural language, reason over contextual information, and effectively rank candidate answers based on their likelihood of being correct. We are provided with separate training and test datasets . The objective is to train models using the provided training data and generate ranked predictions for the unseen test questions.

Model performance is evaluated using **Mean Average Precision at 3 (MAP@3)**, which rewards models that rank the correct answer as highly as possible within their top three predictions. This evaluation metric emphasizes both prediction accuracy and the quality of answer ranking.

Project Objective

The primary objective of this project was to design, implement, and evaluate multiple AI-based approaches for solving the Smart MCQ Solver Challenge. To achieve this, three different models were developed and compared: a **TF-IDF with Logistic Regression**, a scratch **BiLSTM with Attention**, and a fine-tuned **DeBERTa-v3-base** transformer model. Their performance was evaluated using the competition's **MAP@3** metric, along with accuracy and macro F1-score.

The project also provided hands-on experience with modern NLP and deep learning tools, including **PyTorch**, **Hugging Face Transformers**, **Weights & Biases (WandB)** for experiment tracking, and **Gradio** for deploying the final model as an interactive web application.

Report Structure

The report is organized as follows:

- **Section 2** → Dataset & Preprocessing
- **Section 3** → Tokenization Strategy
- **Section 4** → Model Architectures & Training
- **Section 5** → Performance Evaluation & Error Analysis
- **Section 6** →Deployment
- **Section 7** → Conclusion

Dataset & Preprocessing

Dataset

The dataset consists of multiple-choice questions, where each sample contains a **question prompt**, five candidate answer options (**A–E**), and the corresponding correct answer label in the training set. The

objective is to learn the relationship between the question and its candidate answers and predict the most probable answers for unseen test questions.

Exploratory Data Analysis

The prompt length distribution showed that most questions contained between **50 and 200 characters**, indicating a relatively consistent question length suitable for sequence-based models.

Duplicate analysis revealed a significant number of repeated questions within the training set. After removing common instructional prefixes (e.g., *"Choose the correct answer"*), the number of duplicate prompts increased to 1074, indicating that many questions differed only by these template phrases. This observation motivated the removal of instructional prefixes during preprocessing.

A comparison between the training and test datasets further revealed a considerable overlap in question prompts (363). Among the overlapping questions, a large proportion had identical answer options (304), while only a small subset contained modified answer choices.

Preprocessing

Based on the findings from the EDA, a lightweight preprocessing was applied to improve data consistency while preserving the semantic meaning of the questions.

Prompt Cleaning:
Common instructional prefixes, such as *"Choose the correct answer:"*, *"Determine the correct option:"*, and similar variants, were removed from both the training and test datasets using regular expressions.

Label Encoding:
The target labels were converted from categorical answer choices (**A**, **B**, **C**, **D**, and **E**) into integer class indices required for model training.

Tokenization Strategy

Primary tokenizer Selection

The best-performing model in this project was the **BiLSTM with Attention** model, which employed a **custom word-level tokenizer** developed specifically for this architecture. Unlike transformer-based models that rely on pre-trained subword tokenizers, the BiLSTM model was trained entirely from scratch and therefore required a custom vocabulary built directly from the training dataset.

The tokenizer converts all text to lowercase and extracts individual words, numerical tokens, and the special `<SEP>` token using regular expressions. The `<SEP>` token is used to explicitly separate the question prompt from the candidate answer, allowing the model to distinguish between the two input segments.

A simple word-level tokenizer was selected because the BiLSTM model does not depend on pre-trained language representations. Constructed from the training dataset by counting the frequency of each token. `<PAD>` was used to pad all sequences to a fixed maximum length of 128 tokens. For each answer option, the model received input in the following format:

Question <SEP> Candidate Answer

Each token was then converted into its corresponding integer index before being passed to the embedding layer. Fixed-length sequences enabled efficient mini-batch training of the BiLSTM model.

In addition to the custom tokenizer, the project also utilized the **WordPiece tokenizer** provided with the pre-trained **DeBERTa-v3-base** model. WordPiece performs subword tokenization, enabling transformer models to effectively handle unseen or rare words by decomposing them into smaller meaningful units.

The custom word-level tokenizer was selected for the BiLSTM architecture because it aligns naturally with training a model from scratch and introduces minimal preprocessing overhead. DeBERTa model required the WordPiece tokenizer to maintain compatibility with its pre-trained embeddings and transformer architecture.

Modeling & Experimentation

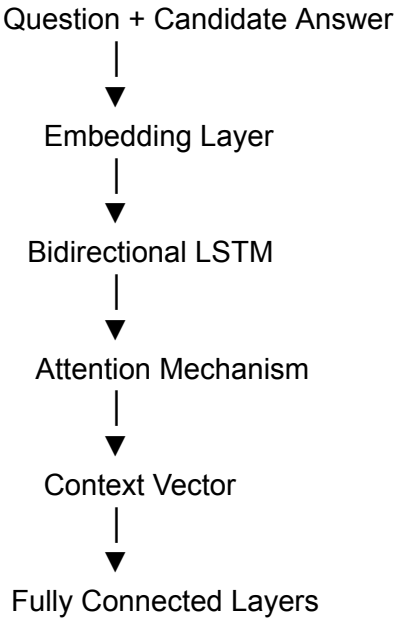
Model 1 - BiLSTM with Attention (Scratch Model)

A custom **Bidirectional Long Short-Term Memory (BiLSTM) with Attention** architecture . For each candidate answer, the question prompt and answer option were concatenated using a special `<SEP>` token. The combined sequence was first converted into dense vector representations using a trainable embedding layer with an embedding dimension of 300. These embeddings were then processed by a two-layer **Bidirectional LSTM**, allowing the model to capture contextual information from both the forward and backward directions of the sequence.

Instead of relying solely on the final hidden state of the BiLSTM, an **attention mechanism** was incorporated to identify and emphasize the most informative words in the sequence. The attention layer computes a weight for every token produced by the BiLSTM encoder and generates a weighted context vector, enabling the model to focus on important keywords while reducing the influence of less relevant words.

The context vector is passed through a fully connected classification network with dropout regularization, producing a single score for each candidate answer. This process is repeated independently for all five answer options, and the resulting scores are compared to determine the most probable correct answer.

Architecture Diagram





Candidate Score

A bidirectional LSTM captures contextual information from both preceding and succeeding words, allowing the model to better understand the relationship between the question and each candidate answer.

The attention mechanism was introduced to overcome the limitation of representing the entire sequence using only the final hidden state. By assigning higher weights to informative tokens, such as important concepts or domain-specific keywords, the model produces a richer representation of the input sequence. This enables the classifier to focus on the most relevant parts of the question-answer pair, resulting in improved answer ranking performance.

The model was trained using an embedding dimension of 300, a hidden size of 256, and **two BiLSTM layers** with 30% dropout for regularization. Training was performed for 15 epochs using the Adam optimizer with a learning rate of 0.001, a batch size of 16, and a maximum sequence length of 128 tokens.

Model 2 - Pre trained model - DeBERTa-v3-base

To leverage the contextual language understanding capabilities of transformer models, **DeBERTa-v3-base** was fine-tuned . DeBERTa (Decoding-enhanced BERT with Disentangled Attention) is a transformer encoder architecture that improves upon the original BERT model by introducing **disentangled attention**, which separately represents the content of a token and its relative position. This enables the model to learn richer contextual representations and capture long-range dependencies more effectively.

For each candidate answer, the input was constructed as a **question-answer pair** in the following format:

Question: Prompt | Candidate Answer: Option

Each of the five candidate answers was processed independently by the tokenizer, and the resulting representations were passed through the pre-trained DeBERTa encoder. A multiple-choice classification head then produced a score for each candidate answer, and the option with the highest score was selected as the final prediction.

The pre-trained **DeBERTa-v3-base** model available through the Hugging Face Transformers library was fine-tuned on the competition dataset. The **AdamW** optimizer was used with a learning rate of 2×10^{-5}, while training was performed for 15 epochs using a per-device batch size of 4 and gradient accumulation of 4 steps to achieve an effective batch size of 16. The maximum sequence length was set to 256 tokens, and label smoothing (0.05) was applied during training to reduce overconfidence and improve generalization.Model performance was monitored using the competition's **MAP@3** metric, and the best-performing checkpoint on the validation set was automatically selected for final evaluation.

Model 3 - TF-IDF with Logistic Regression

As a classical machine learning baseline, a **TF-IDF with Logistic Regression** model was implemented to evaluate the effectiveness of traditional text representation techniques for multiple-choice question answering. Instead of treating the task as a five-class classification problem, each question was transformed into **five independent question-answer pairs**, where each pair represented one candidate answer. The correct

answer was assigned a positive label, while the remaining four candidates were assigned negative labels, converting the problem into a binary classification task.

Text features were extracted using a combination of **word-level TF-IDF** (unigrams and bigrams) and **character-level TF-IDF** (3–5 character n-grams). These feature representations were combined using a **FeatureUnion**, producing a sparse feature vector. The combined feature vector was then passed to a **Logistic Regression** classifier, which estimated the probability of each candidate answer being correct. Finally, the five candidate probabilities were ranked to generate the top three predictions required by the competition.

The model employed **word TF-IDF** with unigram and bigram features, **character TF-IDF** with 3–5 character n-grams, and a maximum vocabulary size of 15,000 features for each representation. The two feature sets were combined using **FeatureUnion**, and a **Logistic Regression** classifier was trained using the **liblinear** solver with a regularization parameter (C) of 4 and a maximum of 2000 iterations.

Performance & Comparative Analysis

Evaluation Metrics

The primary evaluation metric for the competition was **Mean Average Precision at 3 (MAP@3)**, which measures the quality of the ranked predictions. A higher MAP@3 score indicates that the correct answer appears closer to the top of the predicted answer list.

In addition to MAP@3, the models were evaluated using **Accuracy** and **Macro F1-score** on the validation dataset. Accuracy measures the proportion of correctly classified questions, while Macro F1-score evaluates the balance between precision and recall across all answer classes, giving equal importance to each class irrespective of its frequency.

Training Performance

Training progress for the BiLSTM with Attention and DeBERTa-v3-base models was monitored using **Weights & Biases (WandB)**. The recorded loss and validation metrics indicate stable convergence throughout training without significant fluctuations. The best-performing checkpoints were automatically selected based on the validation MAP@3 score and used for final evaluation.

Model	Accuracy	F1	Map @ 3
BiLSTM + Attention	0.9946	0.9946	0.9974
DeBERTa-v3-base	0.9975	0.9974	0.9988
TF-IDF + Lr	0.9900	0.9899	0.9975

Table 1 summarizes the performance of the three models evaluated in this project. The **DeBERTa-v3-base** model achieved the highest validation performance, obtaining a validation accuracy of **99.75%**, a macro F1-score of **0.9974**, and a MAP@3 score of **0.9988**. Its strong performance can be attributed to the rich contextual representations learned through transformer-based self-attention and large-scale pre-training.

The **BiLSTM with Attention** model achieved slightly lower validation metrics but produced the **best Kaggle public leaderboard score of 0.75477**, outperforming the transformer model on the competition test set. The bidirectional sequence encoder, combined with the attention mechanism, enabled the model to effectively capture contextual dependencies while emphasizing the most informative words within each question-answer pair.

The **TF-IDF with Logistic Regression** model served as a strong classical baseline. Despite its simple architecture and significantly lower computational cost, it achieved competitive validation performance, demonstrating that statistical text representations remain effective for multiple-choice question answering when combined with an appropriate ranking strategy.

An interesting observation is that the model achieving the highest validation metrics did not necessarily achieve the highest Kaggle leaderboard score.

Deployment

To demonstrate the practical applicability of the proposed solution, the best-performing **BiLSTM with Attention** model was deployed as an interactive web application using **Gradio**. The trained model weights (`best_bilstm_attention.pth`) and the corresponding vocabulary (`vocab.pkl`) were loaded during inference. User inputs undergo the same preprocessing pipeline used during training, including text normalization, tokenization, numericalization, and sequence padding, ensuring consistency between training and inference. The application performs forward inference on the trained model and returns the predicted answer along with its confidence scores for all candidate options.

Deployment link : dl-genai-26t2-production.up.railway.app

Conclusion

Key Learnings

This project provided practical experience in developing and evaluating multiple approaches for multiple-choice question answering. By implementing a classical machine learning model, a custom deep learning architecture, and a transformer-based model, helped to gain a deeper understanding of the trade-offs between different NLP techniques. The project also strengthened understanding of text preprocessing, tokenization strategies, attention mechanisms, transformer fine-tuning, model evaluation using MAP@3, and experiment tracking using Weights & Biases.

Challenges Faced

The primary challenge was selecting a model that generalized well to the hidden Kaggle test set. Although the DeBERTa model achieved the highest validation performance, it did not produce the best leaderboard score.

Future Work

Ensemble learning techniques combining the predictions of multiple models could improve overall ranking performance.

systematic hyperparameter optimization using frameworks such as Optuna could be employed to identify better training configurations.

Overall, the project demonstrated that different modeling paradigms offer unique advantages for multiple-choice question answering. While transformer models achieved the highest validation performance, the custom BiLSTM with Attention architecture produced the best competition result, emphasizing that model selection should be based not only on architectural complexity but also on empirical performance and generalization.

References

- Shreya Rao medium blogs “Deep Learning Illustrated” https://medium.com/@shreya.rao/list/ae6c27de1640
- Scikit Learning Documentation
- Pytorch documentation
- Hugging Face, "microsoft/deberta-v3-base," Available: https://huggingface.co/microsoft/deberta-v3-base